

Universidad Nacional de Mar del Plata.

Facultad de Ingeniería.

Alumnos:

Bellone, Martín;

García Di Martino, Laureano;

Sorrenti, Enzo;

Paniagua, Lucas;

Zuviría, Nicolás

Programación C

Trabajo Práctico Grupal

Docentes a cargo:

Gellone, Ivonne;

Lazzurri, Guillermo Carlos;

Guccione, Leonel Domingo;

De Lellis, Lucas

Fecha de entrega:

Domingo 16 de Noviembre de 2025.



Introducción

Para este proyecto se requirió el desarrollo de módulos de software para incorporar al sistema de gestión de una pequeña clínica privada. Los módulos solicitados fueron: Facturación al Paciente, Reportes de Actividad de los Médicos (con suma de honorarios), Resolución de Conflictos de Sala de Espera, gestión de Asociados, simulación de uso de una Ambulancia, persistencia de datos y una interfaz gráfica de usuario. Con el objetivo de mantener los módulos escalables y abiertos para su extensión se debieron respetar los principios SOLID de la programación orientada a objetos.

En esta instancia se realizará un resumen de las clases más significativas del modelo planteado, de las decisiones respecto a los módulos mencionados y de las implementaciones de los patrones de diseño requeridos:

- ❖ *Facade*
- ❖ *Decorator*
- ❖ *Double Dispatch*
- ❖ *Observer/Observable*
- ❖ *MVC*
- ❖ *State*
- ❖ *DAO/DTO*

Cabe recalcar que el presente informe no es reemplazo de la lectura de la documentación presente en carpeta JavaDoc.



Descripción de Modelos

Modelo de Aplicación:

El paquete `modeloAplicacion` actúa como una capa de infraestructura encargada de gestionar la comunicación de eventos en tiempo real entre el núcleo del sistema y la interfaz de usuario. Su propósito fundamental es implementar el patrón de diseño Observer para desacoplar la lógica de la simulación de su representación visual. La clase central aquí es `ObservadorSimulacion`, la cual implementa la interfaz `Observer` de Java y mantiene una referencia a la vista (`IVistaSimulacion`). Su función es suscribirse a los objetos activos de la simulación (como la ambulancia o los asociados) e interceptar cualquier cambio de estado. Cuando ocurre un evento, este observador recibe un objeto de la clase `NotificacionSimulacion`, que encapsula tanto el mensaje descriptivo como el tipo de evento (INFO, ALERTA, ERROR). De esta manera, el modelo de aplicación traduce los eventos técnicos del dominio en notificaciones visuales comprensibles, asegurando que la vista se actualice dinámicamente sin que las entidades de negocio conozcan detalles sobre la interfaz gráfica.

Modelo de Dominio:

El paquete `modeloDominio` constituye el núcleo de la lógica de negocio, albergando todas las entidades que modelan el funcionamiento real de la institución médica y sus interacciones. La pieza fundamental de este paquete es la clase `Clinica`, diseñada bajo el patrón Singleton (`Clinica.getInstance()`) para garantizar la existencia de un único punto de acceso centralizado a todos los recursos del sistema, como los registros de médicos, pacientes y habitaciones. Esta clase actúa como un Facade para el Controlador, exponiendo métodos de alto nivel para gestionar ingresos, egresos, reportes y la propia simulación. Dentro de este paquete, `Clinica` orquesta la colaboración entre los distintos subsistemas (como `SistemaIngreso` o `SistemaDeReportes`) y gestiona el ciclo de vida de la simulación, encargándose de iniciar los hilos de ejecución para los asociados y la ambulancia, y de cargar los datos necesarios a través de la capa de persistencia. Es aquí donde residen las reglas "duras" del sistema y las validaciones de integridad de los datos.



Clases más significativas

Clínica:

Esta clase es donde usamos el patrón *Singleton* y *Facade*. Es la clase que contiene atributos como el nombre, la dirección, el teléfono, la ciudad, médicos registrados, pacientes registrados, y sistemas de ingreso, egreso y reportes.

Se usó el patrón *Singleton* para asegurar la existencia de una única instancia con el método *getInstancia*, esto significa que no va a haber más de una clínica.

También se implementó el patrón *Facade* para interactuar de manera más sencilla con los subsistemas que tiene el proyecto. Por medio de los métodos *registrarPaciente*, *registrarMedico*, *ingresarPaciente*, *egresarPaciente*, *atenderPaciente*, *reportesMedicos*, *internarPaciente*. Dichos métodos delegaban la responsabilidad de resolver la implementación a cada módulo correspondiente.

Otros métodos destacables de dicha clase son los métodos *atenderPaciente* y el método *internarPaciente*. En el primero se retira un paciente de la sala de espera y se lo pasa a la lista de pacientes en atención y se registra la consulta para el paciente y para el médico asignado. En el segundo simplemente se registra a un paciente en una habitación y se la marca como ocupada.

Paralelamente la clase Clínica sirve de modelo para el patrón MVC, se comunica con el DAO (gestionando la persistencia), maneja la inicialización y finalización de la simulación, devuelve al controlador DTOs



Paciente:

Para crear los objetos se usó una clase *PacienteFactory*, en ella se crean pacientes según el rango etario (Joven, Mayor o Niño). Estas clases están implementadas usando el patrón *Double Dispatch* para manejar la prioridad en la sala de espera.

Ejemplo en clase Niño:

```
public PacienteNino(/*...atributos de clase persona... */ , String
nroHistoriaMedica, LocalDate fechaIngreso)

{
    super(nombre, apellido, dni, domicilio,
telefono,fechaIngreso,  nroHistoriaMedica);
}

@Override
public boolean prioridad(Paciente paciente){
    return paciente.prioridadConNino();
}

@Override
public boolean prioridadConJoven(){
    return true; // la sala queda para niño
}

@Override
public boolean prioridadConMayor(){
    return false; // el niño va para el patio
}

@Override
public boolean prioridadConNino(){
    return false; // el niño anterior queda en la sala
}
```



Médico:

Para crear este tipo de persona se utiliza la clase *MedicoFactory*, que instancia a los médicos según su especialidad (Cirujano, Clínico o Pediatra), contratación (Permanente o Residente) y posgrado (Magister, Doctorado o de Grado). En esta clase se implementa el patrón *Decorator* para especificar los tipos de contrataciones y posgrados.

El siguiente código ejemplifica nuestra implementación del *MedicoFactory*, el cual instancia al médico en 3 pasos:

- Primero se aplica la Especialidad.
- Luego se decora con el Posgrado.
- Por último, se le aplica la decoración por Contratación.

```
/*
    ...
    variable medico ya instanciada con la especialidad
*/
DecoratorPosgrado decoratorPosgrado = null;
if(titulo.equalsIgnoreCase("magister")){
    decoratorPosgrado = new DecoratorPosgradoMagister(médico);
}
else
    if(titulo.equalsIgnoreCase("doctor"))
    {
        decoratorPosgrado = new DecoratorPosgradoDoctorado(médico);
    }
    else
        throw new TituloNoExistenteException("El tipo de postgrado
ingresado no existe");

DecoratorContratacion decorator = null;
if(contratacion.equalsIgnoreCase("permanente")){
    decorator = new
DecoratorContratacionPermanente(decoratorPosgrado);
}else
    if(contratacion.equalsIgnoreCase("residente"))
    {
        decorator = new
DecoratorContratacionResidente(decoratorPosgrado);
    }
    else
        throw new ContratacionNoExistenteException("El tipo de
contratación ingresada no existe");

return decorator;
```



Habitaciones:

Para las habitaciones se aplicó el patrón *Factory* nuevamente, implementando la clase *HabitacionesFactory* que instancia la habitación solicitada, devolviendo un objeto de tipo *Habitacion*, ya sea privada, compartida o de terapia intensiva. Dichos objetos se encuentran agrupados en un *ArrayList* dentro de la clase clínica.

Ambulancia:

Para la ambulancia se aplicó el patrón *State* ([UML₁](#)), siendo las clases representativas de los estados concretos:

- ❖ *AtendiendoEnDomicilio*
- ❖ *Disponible*
- ❖ *EnTaller*
- ❖ *RegresandoClinicaSinPaciente*
- ❖ *RegresandoDelTaller*
- ❖ *TrasladandoPacienteClinica*

También, la ambulancia es parte de un patrón *Observer* ([UML₂](#)), extendiendo la clase *Observable*. Simultáneamente es el recurso compartido entre los hilos de la simulación, y notifica a observadores cuando la ambulancia se encuentra en un estado en el que no puede cumplir una solicitud. Hace uso de la sentencia *synchronized* en los métodos para que no los usen varios hilos a la vez. También usamos la sentencia *wait()*, para poner a dormir algunos hilos hasta que se cumpla una condición específica, usada para evitar errores en ciertos métodos *synchronized*. En la siguiente captura se ilustra uno de los métodos *synchronized* que notifica a los observadores y a su vez ejecuta una determinada acción de acuerdo con el estado:



```
public synchronized void atenderDomicilio(Asociado a) { 1 usage  Laureano4656 +2
    assert a != null;

    while (this.isOcupado() && this.isSimulacionActiva()) {
        try {
            setChanged();
            notifyObservers(new NotificacionSimulacion(
                mensaje: "❌ 🚑 Ambulancia ocupada, el asociado " + a.getNombre() +
                    " espera para ser atendido a domicilio...", tipo: "NO_AMBULANCIA"));
            wait();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            notifyAll();
            return;
        }
    }

    // Si la simulación terminó mientras esperaba, salir
    if (!this.isSimulacionActiva()) {
        return;
    }
    this.setOcupado(true);
    // Cambiar estado primero
    this.setChanged();
    estadoActual.SolicitudDeAtencionDomicilio();

    // Notificar dentro del synchronized
    setChanged();
    notifyObservers(new NotificacionSimulacion(
        mensaje: "🚑 El asociado " + a.getNombre() + " es atendido a domicilio.", tipo: "OK"));
}
```

Asociado:

Los asociados son parte del patrón *Observer*, extendiendo la clase *Observable*, implementando la interfaz *Runnable* (para simular la concurrencia) y notificando a los observadores cuando y para qué están usando la ambulancia. Son hilos que interactúan con la ambulancia, intentando tomar control del recurso compartido. Las acciones que toman sobre la ambulancia son aleatorias.



```
@Override & Laureano4656 +1
public void run() {
    while (ambulancia.isSimulacionActiva() &&
           this.cantSolicitudesAtendidas < this.maxCantSolicitudes) {

        elegirServicio();

        // Verificar de nuevo después del servicio
        if (!ambulancia.isSimulacionActiva()) {
            break;
        }

        TiempoMuerto.esperar();
        this.cantSolicitudesAtendidas++;
    }

    // Notificar finalización
    this.setChanged();
    this.notifyObservers(new NotificacionSimulacion(
        mensaje: "✅ El asociado " + this.getNombre() + " ha finalizado sus solicitudes. (" +
            this.cantSolicitudesAtendidas + "/" + this.maxCantSolicitudes + ")", tipo: "ASOCIADO"));
}
```

Operario:

El operario también extiende a *Observable* e implementa la interfaz *Runnable*, y notifica cuando está usando la ambulancia para solicitar mantenimiento. El intento de tomar el recurso compartido no es constante, el operario espera cierto tiempo aleatorio luego de solicitar el mantenimiento para competir por el recurso compartido de nuevo. En resumen, es un hilo que se encarga de mandar la ambulancia al mantenimiento cada cierto tiempo.

```
@Override & Laureano4656 +1
public void run() {
    while (this.ambulancia.isSimulacionActiva()) {
        TiempoMuerto.esperar();

        // Verificar de nuevo antes de solicitar mantenimiento
        if (!this.ambulancia.isSimulacionActiva()) {
            break;
        }

        ambulancia.solicitarMantenimiento(o: this);

        // Si la simulación terminó durante el mantenimiento, salir
        if (!this.ambulancia.isSimulacionActiva()) {
            break;
        }

        TiempoMuerto.esperar();
        ambulancia.volviendoDelTaller();

        // Verificar si todos los asociados terminaron
        this.ambulancia.verificarYFinalizarSimulacion();
    }

    // Notificar que el operario finalizó
    this.setChanged();
    this.notifyObservers(new NotificacionSimulacion(
        mensaje: "👤 El operario " + this.getNombre() + " " + this.getApellido() +
            " ha finalizado su trabajo.", tipo: "INFO"));
}
```



Retorno Automático:

Esta clase es vital para la simulación, ya que es una clase que implementa la interfaz *Runnable* y que cada cierto intervalo fijo de tiempo va a intentar tomar control del recurso compartido y va a hacer que retorne la ambulancia a la clínica.

```
@Override  Laureano4656 +1
public void run() {
    while (ambulancia.isSimulacionActiva()) {
        try {
            Thread.sleep(INTERVALO_RETORNO);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return; // Salir limpiamente si el hilo es interrumpido
        }

        // Verificar nuevamente después del sleep
        if (!ambulancia.isSimulacionActiva()) {
            break;
        }

        // Intentar retorno automático
        // El método retornoAutomatico en Ambulancia debe verificar
        // si es apropiado hacer el retorno según el estado actual
        ambulancia.retornoAutomatico(this);
    }
}
```



Descripción de Módulos

Módulo de Ingreso:

Este módulo se encarga de manejar la incorporación de un paciente a la clínica llevándolo a la sala de espera, en esa clase se realiza el análisis para saber si el paciente irá a la sala de espera privada o al patio. En este proceso se utiliza el patrón *Double Dispatch* para la prioridad de los pacientes, de acuerdo con las especificaciones dadas.

Además, se encarga de sacar los pacientes de la sala de espera y devolver un valor booleano según la respuesta dependiendo si se logró retirar al paciente de la sala de espera o no. Esta respuesta la usa el método *atenderPaciente* para manejar la atención.

Módulo de Egreso:

Este módulo gestiona la generación y emisión de la factura, permitiendo la salida del paciente tras el egreso.

La factura se genera a través del método *egresarPaciente* de la clase *SistemaDeEgreso*. Este método utiliza los datos del paciente y su historial de atenciones/consultas para crear la factura.

La factura tiene un paciente asignado, con relación de composición, un número de factura y un campo detalle que tiene la información sobre las consultas realizadas y el costo final de la misma. Asimismo, previo a retornar una factura, el método de *egresarPaciente* se encarga de eliminar las consultas realizadas por el paciente, para que en caso de un reingreso a la clínica no se cuenten las consultas ya realizadas a la nueva factura. Cabe aclarar que estas consultas se eliminan únicamente del lado del paciente, el médico que lo atendió mantiene registro de dichas consultas para luego poder retornar su reporte.

Módulo de Reportes:

Este módulo se encarga de llevar un registro de las consultas médicas realizadas por cada paciente y, simultáneamente, otro registro de pacientes atendidos por cada médico. Este sistema actualiza dichos registros de cada vez que un médico atiende a un paciente, y elimina las consultas realizadas por un paciente cada vez que este egresa, usando un *HashMap* de *ArrayList* para cada uno.



Controlador:

El Controlador es el componente central del patrón MVC ([UML₃](#)) en nuestra aplicación. Implementa la interfaz `ActionListener` para centralizar todos los eventos de la UI (como clics de botón) provenientes de `IVistaPrincipal` e `IVistaSimulacion`.

Su método `actionPerformed` actúa como un despachador (dispatcher), traduciendo los comandos de acción de la vista (ej. `DAR_ALTA`) en llamadas a métodos de lógica específicos (ej. `guardarAsociado()`).

Estos métodos orquestan el flujo completo de la aplicación: (1) obtienen los datos de entrada desde la Vista, (2) los envía al Modelo (Clinica) para que ejecute la lógica de negocio, y (3) actualiza la Vista con el resultado (ej. refrescando la lista de asociados o mostrando un mensaje de éxito).

Además, el controlador maneja las excepciones (como `SQLException` o `datosAsociadoDTOIncorrectoException`) que pueden surgir tanto en la vista como en el modelo, asegurando que los errores se comuniquen al usuario de forma controlada a través de la interfaz gráfica.

```
@Override Lucas Paniagua +2
public void actionPerformed(ActionEvent e) {
    String comando = e.getActionCommand();
    switch (comando) {
        case IVistaPrincipal.ACCEPTAR: {
            this.iniciarSimulacion();
            break;
        }
        case IVistaPrincipal.DAR_ALTA: {
            this.guardarAsociado();
            break;
        }
        case IVistaPrincipal.DAR_BAJA: {
            this.eliminarAsociado();
            break;
        }
        case IVistaSimulacion.FINALIZAR_SIMULACION: {
            this.finalizarSimulacion();
            break;
        }
    }
}
```



Vista

Vista: Ventana Principal

La clase `VentanaPrincipal` constituye la interfaz gráfica central de la aplicación e implementa la interfaz `IVistaPrincipal`. Su estructura visual se organiza mediante un `JTabbedPane` que divide la funcionalidad en dos módulos claros: la gestión administrativa de asociados y la configuración de parámetros para la simulación. Más allá de la presentación, esta clase desempeña un rol crítico como primera barrera de validación de datos. Al recuperar la información ingresada por el usuario a través de métodos como `getNuevoAsociado()` o `getCantAsociados()`, la vista aplica reglas de integridad rigurosas — como verificar campos vacíos o asegurar valores positivos— y lanza excepciones propias como `datosAsociadoDTOIncorrectoException` si la entrada no es válida, impidiendo así que datos corruptos lleguen al modelo. Para mantener el desacoplamiento con la lógica de negocio, la comunicación con el Controlador se realiza mediante `ActionCommands` (cadenas estáticas como `DAR_ALTA` o `ACEPTAR`) y poniendo al controlador con el *listener* de todos los elementos que producen eventos, permitiendo que la interfaz notifique las intenciones del usuario sin conocer la implementación subyacente. Asimismo, su rol de visualización es pasivo, limitándose a refrescar su `DefaultListModel` interno únicamente cuando recibe la orden explícita a través de `actualizarListaAsociados()`.

```
@Override 1 usage  Enzo Sorrenti +1
public void setActionListener(Controlador controlador) {
    assert controlador != null;
    this.darDeAltaButton.addActionListener(controlador);
    this.darDeBajaButton.addActionListener(controlador);
    this.aceptarConfiguracionButton.addActionListener(controlador);
}
```

Vista: Ventana de Simulación

Por su parte, la clase `VentanaSimulacion` implementa la interfaz `IVistaSimulacion` y cumple una función especializada como monitor de actividad en tiempo real. A diferencia de la ventana principal, esta vista gestiona su propio ciclo de vida visual, haciéndose visible y tomando el foco solo cuando el Controlador invoca `iniciarSimulacion()`, y ajustando el estado de sus controles (como deshabilitar el botón de finalizar) una vez que el proceso concluye. Su característica técnica más relevante es la capacidad de interpretar semánticamente el flujo de la simulación. El método `actualizarEstadoSimulacion()` recibe objetos `NotificacionSimulacion` y, en lugar de simplemente concatenar texto, evalúa el tipo de notificación para renderizar el mensaje con formato enriquecido en un `JTextPane`. Mediante el uso de `StyledDocument`, la vista asigna dinámicamente colores específicos



según la gravedad del evento —azul para información, verde para éxito ("OK") y naranja para alertas ("NO_AMBULANCIA")— proporcionando así un feedback visual inmediato y jerarquizado sobre el estado de la atención clínica.

```
@Override 1 usage 2 Enzo Sorrenti +1
public void setActionListener(Controlador controlador) {
    assert controlador != null;
    this.controlador = controlador;
    this.finalizarButton.addActionListener(controlador);

    // También agregar listener para Enter en el botón
    finalizarButton.addActionListener( ActionEvent e -> {
        // System.out.println("Botón Finalizar presionado - Comando: " + e.getActionCommand());
    });
}
```

Persistencia

Base de Datos:

La clase BaseDeDatos funciona como el motor de bajo nivel de nuestra capa de persistencia, encargándose de gestionar la conexión directa con el servidor MySQL mediante JDBC. Para garantizar una gestión eficiente de los recursos, esta clase implementa el patrón de diseño Singleton (accesible a través de `getInstancia()`), lo que asegura que exista una única instancia de conexión compartida durante toda la ejecución del programa. Más allá de las operaciones estándar de ejecución de consultas (`ejecutarConsulta`, `ejecutarInsert`), esta clase incluye una funcionalidad crítica en su método `iniciarBD()`: al arrancar, se encarga de preparar el entorno ejecutando scripts DDL para crear la estructura de tablas desde cero e inyectar un set inicial de datos de prueba (con apellidos curiosos basados en patrones de diseño como "State" o "Singleton"), facilitando así las pruebas inmediatas sin configuración manual previa.



```
private void abrirConexion() { 1 usage 2 mbell
    try {
        conexion = DriverManager.getConnection(URL, USUARIO, CONTRASENA);
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

public void ejecutarInsert(String insert, AsociadoDTO aDTO) throws SQLException { 1 usage 2 Enzo Sorrenti +2
    assert insert != null;
    assert aDTO != null;

    PreparedStatement stmt = conexion.prepareStatement(insert);
    stmt.setString(parameterIndex: 1, aDTO.getNombre());
    stmt.setString(parameterIndex: 2, aDTO.getApellido());
    stmt.setString(parameterIndex: 3, aDTO.getDni());
    stmt.setString(parameterIndex: 4, aDTO.getCalle());
    stmt.setInt(parameterIndex: 5, aDTO.getNumero());
    stmt.setString(parameterIndex: 6, aDTO.getCiudad());
    stmt.setString(parameterIndex: 7, aDTO.getTelefono());
    stmt.executeUpdate();
    stmt.close();
}
```

Data Access Object:

La clase DataAccessObject actúa como la capa de abstracción que aísla a la lógica de negocio de los detalles del lenguaje SQL. Su responsabilidad principal es traducir las intenciones del modelo (como guardar o eliminar un asociado) en sentencias SQL concretas (como INSERT INTO o DELETE FROM) que luego delega a la clase BaseDeDatos. Este componente realiza el mapeo objeto-relacional manual: toma objetos de dominio Asociado, los convierte en objetos de transferencia AsociadoDTO mediante métodos de utilidad como CrearDTO, y viceversa al leer resultados de la base de datos. Además, el DAO implementa validaciones defensivas utilizando assert para asegurar que no se intenten persistir objetos nulos o realizar consultas con IDs inválidos antes de siquiera contactar a la base de datos.

```
public AsociadoDTO CrearDTO(int id, String nombre, String apellido, String dni, String calle, int numero, String ciudad, String telefono) {
    AsociadoDTO aDTO = new AsociadoDTO();
    aDTO.setNombre(nombre);
    aDTO.setApellido(apellido);
    aDTO.setDni(dni);
    aDTO.setTelefono(telefono);
    aDTO.setCalle(calle);
    aDTO.setNumero(numero);
    aDTO.setCiudad(ciudad);
    return aDTO;
}
```



```
public ArrayList<AsociadoDTO> cargarBD() throws SQLException { // usage: &mbell *
    String query = "SELECT * FROM Asociados ";

    ResultSet resultado = bd.ejecutarConsulta(query, id: -1);
    ArrayList<AsociadoDTO> asociadosDTO = new ArrayList<>();
    while (resultado.next()) {
        AsociadoDTO aDTO = this.CrearDTO(resultado.getInt(columnLabel: "id"), resultado.getString(columnLabel: "nombre"), resultado.getString(columnLabel: "apellido"),
            resultado.getString(columnLabel: "dni"), resultado.getString(columnLabel: "calle"), resultado.getInt(columnLabel: "numero"),
            resultado.getString(columnLabel: "ciudad"), resultado.getString(columnLabel: "telefono"));
        asociadosDTO.add(aDTO);
    }
    resultado.close();
    return asociadosDTO;
}
```

Asociado DTO:

Finalmente, la clase AsociadoDTO (Data Transfer Object) representa la estructura de datos pura que viaja entre las capas de la aplicación. A diferencia de los objetos del modelo de dominio, que pueden tener comportamiento y lógica compleja, este DTO es un contenedor simple (POJO) que implementa la interfaz Serializable, permitiendo que los datos del asociado puedan ser transmitidos o almacenados si fuera necesario. Su diseño incluye constructores flexibles y métodos *setters* que incorporan aserciones (assert) para validar la integridad básica de los datos —como evitar nombres nulos o números negativos— asegurando que la información que llega a la capa de persistencia o a la vista cumpla con los requisitos mínimos de consistencia.



Anexo





